SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

Teapot is an AI-powered business intelligence and consultation platform developed as a strategic solution to the challenges identified in the problem statement. Many aspiring entrepreneurs and small business owners face difficulties in transforming abstract ideas into structured, realistic, and actionable business plans due to the complexity of market analysis, risk evaluation, and decision-making. While existing AI tools provide general information, they often remain at a theoretical level and lack structured guidance tailored to real-world business execution.

To address this gap, Teapot is designed as a centralized web-based system that integrates multiple specialized AI agents powered by advanced language models such as Z.AI GLM, Google Gemini, and Anthropic Claude. These agents work collaboratively to simulate expert-level business consultation by analysing user inputs, identifying market opportunities, evaluating risks, optimizing resources, and generating strategic recommendations. The system also supports scenario simulation and automated document generation, enabling users to convert ideas into structured business outputs such as business plans and pitch decks.

By combining multi-agent intelligence with an interactive chat-based interface and project management features, Teapot aims to bridge the gap between business theory and practical execution. It provides users with a guided, data-driven environment where ideas can be validated, refined, and developed into realistic business strategies, ultimately empowering users to make more confident and informed entrepreneurial decisions.

© UMHackathon 2026

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the development of the Teapot, which is designed to assist entrepreneurs in transforming business ideas into structured, data-driven strategies using multi-agent AI systems.

So, the key elements of system that has been covered are as follows:

1. **Architecture**:
    a. Web-based platform built using **FastAPI (backend) and Next.js (frontend)**
    b. Integration with multiple LLMs (**Z.AI GLM, Google Gemini, Anthropic Claude**)
    c. Modular system design with separated components (AI agents, user management, document generation)
    d. AI agent framework includes:
        - Sales Predictor
        - Pain Point Analyser
        - Profit Optimiser
        - Decision Recommendation
        - Risk Identifier
        - Scenario Simulator
        - Resource Optimizer
    e. Supports **custom AI agent creation by users** for specialized business needs
    f. Designed for scalability and extensibility
2. **Data Flows:**
    a. User inputs business ideas or queries via conversational interface
    b. System performs intent analysis and routing to relevant AI agents
    c. Default or custom agents process requests independently
    d. Outputs are aggregated into a unified response
    e. Data is stored in PostgreSQL for project tracking and history
    f. Final results returned as insights or generated documents (PDF/.CSV)
3. **Model Process:**
    a. User input triggers one or more default or custom AI agents
    b. Each agent performs domain-specific analysis (e.g., risk, profit, scenario simulation)
    c. Outputs from multiple agents are combined into a single recommendation
    d. Supports iterative refinement through user feedback loops

© UMHackathon 2026

   e. Enables continuous improvement of business strategies within a project
4. Role of Reference:
   a. Acts as a technical guide for developers, testers, and stakeholders
   b. Provides overview of system architecture and AI agent design
   c. Documents data flow and model interaction logic
   d. Ensures consistent understanding for development, scaling, and maintenance

1.2. Background:

The development of the Teapot is driven by the increasing complexity and uncertainty faced by aspiring entrepreneurs and small business owners when starting or managing a business. Many users rely on fragmented tools, static business templates, or generic AI chat systems that lack structured reasoning and domain-specific analysis. This results in difficulty in validating ideas, assessing risks, and translating concepts into actionable business strategies.

1. Previous Version:
   a. In traditional approaches, business planning is often done using manual methods such as spreadsheets, static templates, or consultancy-based services.
   b. Existing AI solutions typically operate as general-purpose chatbots, providing broad advice without structured business decomposition.
   c. Some platforms offer basic tools like idea brainstorming or document generation, but they lack multi-step reasoning and specialized analysis for different business domains.
   d. There is no unified system that combines idea validation, risk analysis, profit optimization, and scenario simulation in a single workflow.
2. Changes in Major Architectural Components
   a. Transition from single-model or general chatbot systems to a multi-agent AI architecture
   b. Introduction of specialized AI agents (Sales Predictor, Risk Identifier, etc.) working collaboratively
   c. Shift from isolated tools to a centralized business intelligence platform with project management support
   d. Integration of multiple LLM providers (Z.AI GLM, Gemini, Claude) for diversified reasoning capabilities
   e. Addition of custom AI agent creation, allowing users to extend system functionality based on their needs
3. New capabilities that are only introduced in this

© UMHackathon 2026

a. Multi-agent collaborative business analysis instead of single-response AI outputs
b. Real-time scenario simulation and business forecasting
c. Structured risk, profit, and resource optimization analysis
d. Automated generation of business documents (business plans, pitch decks, reports)
e. Interactive project-based workspace for managing multiple business ideas
f. Ability for users to create and customize their own AI agents for specialized tasks
g. End-to-end transformation from idea → analysis → strategy → execution-ready output

1.3. Target Stakeholder

Stakeholders	**Roles**	**Expectations**
Entrepreneurs / Business Users (Customer)	Input business ideas, interact with AI agents, and generate business insights, strategies, and documents through the platform.	Simple and intuitive experience to transform ideas into structured business plans, receive data-driven recommendations, and access scenario-based analysis without requiring technical or business expertise.
Startup Founders / Small & Medium Business Owners	Use the system for validating business ideas, optimizing operations, and making strategic decisions.	Reliable AI-driven insights for decision-making, risk evaluation, profit estimation, and scalable business planning support.
System Administrators	Manage users, AI models, agents configuration, and monitor system usage (tokens, performance, logs)	Full control dashboard, system monitoring tools, secure authentication system, and ability to configure or update AI agent behavior efficiently

© UMHackathon 2026

Development Team	Build the platform and maintain it	Clear API usage contracts, codebase is modular and well-documented architecture
QA Team	Validating the system behavior for all the user roles	Clearly defined system behavior, consistent response formats from AI agents, and comprehensive coverage of edge cases including invalid inputs and model failures

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Application
Architecture	Cloud-based modular multi-service architecture is integrated with multi-LLM AI agent system.

The Teapot AI Business Intelligence Platform is a web-based system where users interact through a Next.js frontend for business consultation, project management, and AI-driven insights. The frontend communicates with a FastAPI backend that handles authentication, request processing, and AI agent orchestration.

The backend integrates multiple LLMs, including Z.AI GLM, Gemini, and Claude, to power specialized AI agents. Core modules include AI orchestration, user management, project handling, and document generation.

All data is stored in a centralized PostgreSQL database, covering user data, projects, and AI outputs. The system is cloud-deployed to ensure scalability, reliability, and future expansion of AI agents.

© UMHackathon 2026

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic "AI Module". The following Dependency diagram includes:

2.1.3 Dependency Diagram

a. How the prompts are being constructed and sent to the GLM API
b. What goes into the context window?
c. How is your system receiving, parsing the responses and passing to the next system component?
d. Where the limitation of token are enforced or input are chunked before it actually reaches to GLm
e. Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls "Amazon API Gateway" for Order Service]
f. Do show how frontend, backend, services, database and external API interacts with each other [e.g. Order service communicates with Amazon RDS to ensure persistence of order data]
g. Must highlights the distributed components if the system is planning to be deployed across multiple services

2.2.2. Sequence Diagram

a. Here, do walk through one user interaction flow or customer journey with the stakeholders [e.g. Place Order Flow]
b. Show the interactions step by step.
c. Do demonstration of each key system features does operate from start to finish

© UMHackathon 2026

2.2. Technological Stack

2.2.1. Frontend:

Next.js (React Framework) – Used to build a responsive and interactive web-based user interface for business consultation, AI chat interaction, and project management.

2.2.2. Backend

FastAPI (Python) – Serves as the main backend framework providing RESTful APIs for user requests, AI agent orchestration, authentication, and data processing.

2.2.3. Database

PostgreSQL – Relational database used to store user accounts, project data, AI-generated outputs, chat history, and system logs.

2.2.4. Cloud/Deployment

Render – Used as the cloud deployment platform for hosting the FastAPI backend and supporting services. It provides automated deployment, scalability, and continuous integration from the code repository.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

a. Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases

b. Show how proper laws are met (e.g. Black Hole)

2.3.2. **Normalized Database Schema**

a. Show the logical data model (ERD) with Database designed to 3NF form to ensure data integrity and relationships. [e.g. order items is a junction table that is resolving the M: N relationship between orders and menu items.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

© UMHackathon 2026

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. **[e.g. in Table]**

#	Features	Description
1	AI Business Idea Chat Interface	Users can input business ideas or problems in natural language and interact with the system through a conversational interface to receive structured AI feedback.
2	Multi-Agent Business Analysis System	The system activates the 7 core AI agents (Sales Predictor, Pain Point Analyser, Profit Optimiser, Decision Recommendation, Risk Identifier, Scenario Simulator, Resource Optimizer) to generate specialized insights.
3	Scenario Simulation (Basic Version)	Users can adjust simple business assumptions (e.g., cost, demand) and view comparative outcomes (best/worst/expected cases).
4	Project Saving & History Tracking	Users can save business ideas and revisit previous AI-generated analyses within a centralized project workspace.

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality	**Requirements**	**Implementation**
Scalability	The system must support multiple concurrent users interacting with AI agents without significant performance degradation.	Deployed on Render cloud infrastructure with scalable backend services and efficient API routing to LLM providers
Reliability	AI-generated responses must remain consistent and system should avoid service breakdown during model/API failure.	Implemented multi-LLM fallback mechanism (Z.AI GLM, Gemini, Claude) and retry logic for failed responses.
Maintainability	System must allow easy updates to AI agents, models, and backend services.	Modular architecture separating AI agents, backend services, and frontend, enabling independent updates and CI/CD deployment via Render.
Token Latency	AI responses should be returned within **3–5 seconds under normal load (p95 target)**.	Uses async API calls, response streaming, and prompt optimization to reduce LLM processing time.
Cost Efficiency	Average token usage per session must remain within controlled budget limits.	Implements **input chunking, response summarization, caching of repeated queries (24h), and selective agent triggering** to reduce unnecessary LLM calls.

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

a. **Real-time In-App Communication System**

- Direct chat between users and AI agents, as well as potential future support for collaboration between users (e.g., co-founders or teams working on the same project).

b. **Automated Market Data Integration (Live Market Feeds)**

- Real-time integration with external market APIs to continuously update AI analysis with live trends, pricing, and competitor data.

c. Advanced Financial Planning & Forecasting Module

- Full financial modelling system including cash flow projection, balance sheets, and long-term investment forecasting.

d. **Automated Team Formation System**

- AI-driven matching of users based on skills, interests, and project needs to form startup teams automatically.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

The system applies a controlled deployment strategy using grayscale rollout and A/B testing to minimize risk and optimize performance. New features (e.g., new AI agents or prompt strategies) are first released to **5–10% of users**, where key metrics such as **response latency (target <5s), error rate (<5%), and hallucination rate (<10%)** are monitored. If performance remains stable, rollout gradually increased to **50% and then 100%**. A/B testing is conducted by comparing different prompt structures or model combinations (e.g., Z.AI GLM vs fallback models) to determine which produces more accurate and usable outputs.

© UMHackathon 2026

Emergency Rollback (ER) & Golden Release: In case of anomalies, an **Emergency Rollback (ER)** is triggered, reverting the system to a **Golden Release** (last stable version within 24 hours). A priority matrix governs response actions: critical failures (e.g., system downtime or >20% API failure rate) trigger immediate rollback, while lower-priority issues are logged and resolved in subsequent updates.

4.1.2. Priority Matrix:

4.1.2.1. Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project. [e.g. P1 Critical: If payment failure error is more than 1%, emergency rollback to Golden release.]

4.2. Monitoring:

The system includes continuous monitoring mechanisms to ensure performance, reliability, and quality of AI outputs. Key metrics such as API response time, token usage, system uptime, and error rates are actively tracked. In addition, the system monitors AI response quality, including detection of off-topic or unusable outputs. Logging and alerting mechanisms are implemented to identify abnormal patterns such as latency spikes or increased failure rates. When predefined thresholds are exceeded, alerts are triggered to enable rapid intervention, including switching to fallback models or initiating rollback procedures to maintain system stability.

4.3. Assumptions:

The system assumes that users have stable internet connectivity to interact with the web-based platform and that external LLM services, including Z.AI GLM, Gemini, and Claude, are available and responsive. It is also assumed that users provide relevant and structured business-related inputs to ensure meaningful AI-generated insights. The platform operates in a cloud environment (Render), which is expected to provide consistent uptime and performance. Additionally, it is assumed that token usage per session remains within manageable limits, allowing the system to balance performance and operational cost effectively.

© UMHackathon 2026

4.4. External Dependencies: Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
Z Ai GLM API	Core LLM reasoning engine for structured business analysis, multi-step prompting, and generating actionable insights across AI agents.	High – API downtime or rate limits will impact core functionality; mitigation includes fallback models and graceful degradation
Google Gemini API	Secondary LLM for enhanced reasoning, alternative responses, and fallback support	Medium – Service instability may affect fallback reliability; mitigated by multi-model redundancy
Anthropic Claude API	Supports conversational quality, safe outputs, and alternative reasoning paths	Medium – API limits may reduce response quality diversity; fallback to other models available
Render (Cloud Platform)	Hosts backend services, handles deployment, scaling, and uptime management	Medium – Service outage may affect system availability; mitigation includes monitoring and redeployment strategies
PostgreSQL (NeonDB)	Stores user data, project data, AI outputs, and system logs	High – Database failure may lead to data loss or inaccessibility; mitigated via backups and managed service reliability
OAuth Providers (Google / GitHub)	Enables secure user authentication and login	Low – Temporary login issues if provider is down; fallback

© UMHackathon 2026

		to standard login if implemented

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline: Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]

Day 1-2	Define the tech stack, system requirements and the task divisions
Day 3-4	Design Database Schema and User Interface
Day 5-8	Coding and Deploy
Day 9-10	Documentation

5.2. Team Members Role:

Alexander Leslie Cheng – Responsible for the design and development of the AI agent's backend, including multi-agent orchestration, LLM integration, and prompt handling logic.

Chen Chor Yan – Led the frontend design and development, focusing on user interface, user experience, and integration with backend services.

Chung Jing Jia – Developed backend functionalities including CRUD operations and user authentication

Chong Zhen Yi – Develop the chat system and leaderboard features

© UMHackathon 2026

5.3. Recommendations:

To improve scalability, performance, and reliability of the Teapot platform, several enhancements are recommended. For scalability, implement horizontal scaling on Render and introduce a task queue system (e.g., Celery or background workers) to handle AI agent processing asynchronously, reducing load on the main API. For performance, integrate caching mechanisms (e.g., Redis) to store repeated queries and AI outputs, minimizing redundant LLM calls and reducing latency and cost. Additionally, optimize prompt structures and limit unnecessary agent execution through smarter routing logic.

For reliability, adopt a more robust multi-LLM fallback strategy with automated switching between Z.AI GLM, Gemini, and Claude based on failure or latency thresholds. Implement circuit breaker patterns to prevent cascading failures when external APIs are unstable. Database reliability can be improved through automated backups and read-replica support. Finally, enhance monitoring with real-time alerts and logging dashboards to proactively detect anomalies, ensuring consistent system stability and better user experience.

© UMHackathon 2026